

CS F407: Logical Planning

AI Laboratory

Tasks 0–1: Specification and manual plan

Let $R_\ell = At(Robot, \ell)$, $P_\ell = At(Package, \ell)$, and $H = Holding(Package)$. Initially $\{R_A, P_A\}$ holds; the goal is $\{P_C\}$. Connections are $A \leftrightarrow B \leftrightarrow C$.

Action	Preconditions	Delete	Add
$Move(x, y)$	$R_x, Connected(x, y)$	R_x	R_y
$PickUp(x)$	$R_x, P_x, \neg H$	P_x	H
$Drop(x)$	R_x, H	H	P_x

Only connected movement actions are instantiated. Initially $Move(A, B)$ and $PickUp(A)$ are applicable. $Drop(C)$ is not, because R_C and H are absent. The negative pickup condition makes the one-package assumption explicit.

Step	Action	State
0	Initial	$\{R_A, P_A\}$
1	PickUp(A)	$\{R_A, H\}$
2	Move(A,B)	$\{R_B, H\}$
3	Move(B,C)	$\{R_C, H\}$
4	Drop(C)	$\{R_C, P_C\}$

The handout’s illustrative $Move(A, B), PickUp(B)$ prefix is invalid from this initial state: moving leaves the package at A.

Tasks 2–4: Planner and validation

States are immutable sets of propositions. Applicability requires all positive preconditions and no negative preconditions. Applying an action gives $S' = (S \setminus Delete(a)) \cup Add(a)$. BFS explores unvisited successors with a FIFO queue. The goal test checks inclusion of all goal facts; predecessors recover the shortest plan. An exhausted queue means no plan. The finite fact set and visited-state tracking prevent endless exploration.

Test	Plan found	Actions	Replay valid
Original	Yes	4	Yes
No pickup actions	No	–	Not applicable
Added irrelevant Wait	Yes	4	Yes
Movement actions only	No	–	Not applicable

Movement-only actions verify that the robot reaching C is insufficient. A no-effect Wait checks that irrelevant actions do not change the plan. Replay checks applicability, one robot location, and exactly one package location or holding state. Other checks cover an already satisfied goal and a violated negative precondition. The missing flowchart step is applying action effects. Logic determines legal successors; search chooses their sequence. The optional LLM plan explanation should agree with independently replayed transitions.

Reflections

Preconditions and effects specify legality and state changes. Omitting a condition could permit pickup from a different location or dropping an unheld package. A plausible sequence can therefore fail. The LLM supplied the planner; replay and known-answer tests verified it. Logic is used in applicability and goal checks; planning searches propositional states.

Optional Prolog extension

`planner.pl` includes the facts, rules, and six query checks. **TODO:** Run the checks in `planner.pl` with SWI-Prolog. Expected logical outcomes are: `can_move(a,b)` true, `can_move(a,c)` false; `valid_move(a,b)` and `valid_move(b,c)` true; `valid_move(a,c)` false. The last query fails because the rule checks direct connections, not transitive reachability. `wet_road  $\Rightarrow$  slippery  $\Rightarrow$  reduce_speed` proves the final query. A fact is a base assertion; a rule derives an assertion; a query asks whether rules establish it. An independent verifier can reject invented movement edges. These rules do not verify full package-carrying plans. Prolog’s unification, backtracking, and failure behaviour should not be identified with all of classical logic.

Reproduce. Run `../venv/bin/python solution.py` here. Results: `results.json`; prompt and correction record: `prompts.txt`.